JP MORGAN CHASE · CIBAP PLATFORM

MCP Server Architecture & Design

How the Model Context Protocol Gateway translates AI tool calls into real API requests across the CIBAP, CSIP, and CTIP platforms — and what connects everything together.

Platform	Port	Total Tools	Date
cibap-mcp-server	8095	30 (8 + 10 + 12)	April 2026

Contents

1. The World This MCP Server Lives In
2. What the MCP Server Actually Is
3. Architecture Diagram
4. How a Request Flows Through the Code
5. The Three Clients and What They Know
    1. CibapClient — Five Services, One Client
    2. CsipClient — Support Intelligence
    3. CtipClient — Test Intelligence
6. Configuration: The Single Source of Truth
7. Security and Error Handling
8. The Tool Catalogue
9. Full Tool Inventory (30 tools)

1. The World This MCP Server Lives In

The CIBAP platform is a collection of seven running services spread across your local environment. Five of them are Java Spring Boot microservices that handle real banking domain work. One manages customer data and support requests, backed by Spring WebFlux and R2DBC. One scores transactions for fraud in real time using Drools rules combined with a SageMaker ML model. One runs a six-state loan pre-screening workflow managed by Spring State Machine. One dispatches notifications via Kafka and SNS/SES. And one is an AI orchestration service powered by LangChain4j and Amazon Bedrock Claude. Those five services sit on ports `8081` through `8085`.

Then there are two newer Python FastAPI services added to the project. One is **CSIP** — the CIBAP Support Intelligence Platform — running on port `8092`. It monitors deployed applications for issues, detects problems autonomously, opens fix pipelines, and maintains a full audit log of every action it takes. The other is **CTIP** — the CIBAP Test Intelligence Platform — running on port `8091`. It manages test cases with semantic search, runs test suites in parallel, evaluates deployment gates, tracks flaky tests, and operates a "web scout" that continuously watches for new test scenarios.

None of these seven services knows anything about Claude or AI tooling. They are plain HTTP services that speak ordinary REST. That gap — between an AI assistant and a fleet of domain services — is exactly what the MCP server bridges.

2. What the MCP Server Actually Is

The `cibap-mcp-server` is a translator. It sits on port `8095` and connects an AI assistant (Claude) to those seven services using a protocol called the **Model Context Protocol (MCP)**.

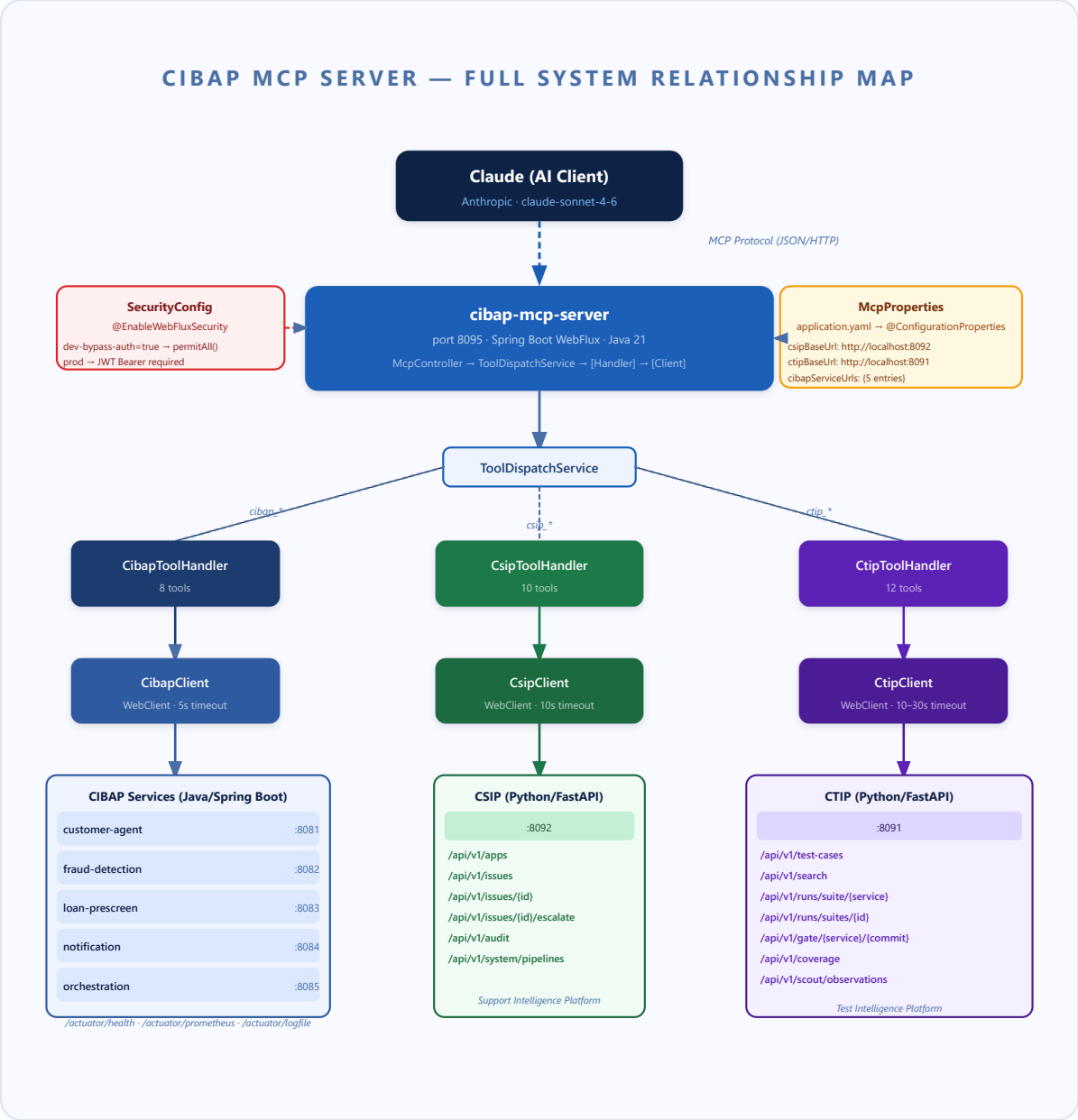
MCP is a simple JSON contract with two sides. On one side, an AI client asks "*what can you do?*" and receives back a list of named **tools** — each with a description, typed input parameters, and a flag indicating whether calling it has side effects. On the other side, the AI client says "*call this tool with these arguments*" and receives a structured result it can reason about and present to a user.

The MCP server exposes exactly **two HTTP endpoints** to the outside world:

Endpoint	Purpose
`GET /api/v1/mcp/tools`	Returns the catalogue of all 30 tools as a JSON array
`POST /api/v1/mcp/tools/call`	Accepts a tool name and arguments, executes the tool, returns the result
`GET /`	Status page — service name, tool count, endpoint listing
`GET /actuator/health`	Spring Boot Actuator health check for the MCP server itself

When Claude is configured with this MCP server's URL, it loads the tool list once at startup. From that point forward, Claude can autonomously decide to call `cibap_health_check` to check if services are running, or `csip_get_issues` to find open production problems, or `ctip_run_suite` to trigger a test run — all by making HTTP calls to this single gateway on port 8095. Claude never directly reaches port 8081, 8082, 8091, or 8092. It only ever talks to the MCP server.

3. Architecture Diagram



4. How a Request Flows Through the Code

To make this concrete, walk through a real call. Claude posts the following to the server:

```
POST http://localhost:8095/api/v1/mcp/tools/call

{
  "name": "csip_get_issues",
  "arguments": {
    "status": "open",
    "severity": "critical"
  }
}
```

1. **Controller — McpController.** The request enters `McpController`, a standard Spring WebFlux `@RestController`. It deserialises the body into a `McpToolCallRequest` record (a Java 21 record with two fields: `name` and `arguments`). The `@Valid` annotation triggers Jakarta Bean Validation — if `name` is blank, a 400 is returned immediately without touching any downstream service.

2. **Dispatch — ToolDispatchService.** The controller hands the validated request to `ToolDispatchService`. This component inspects the tool name's prefix and branches: `cibap_` → CibapToolHandler, `csip_` → CsipToolHandler, `ctip_` → CtipToolHandler. Anything else returns an immediate failure response without making any downstream call. For our example, `csip_` routes to `CsipToolHandler`.

3. **Handler — CsipToolHandler.** The handler's job is argument translation, nothing more. It pulls `status`, `category`, `severity`, `service`, and `limit` out of the arguments map, normalises the string `"all"` to `null` (which means "no filter" to the client), and calls `csipClient.getIssues(...)`. The handler knows nothing about HTTP.

4. **Client — CsipClient.** The client holds a Spring `WebClient` whose base URL is `http://localhost:8092`. It constructs the URI with query parameters, calls CSIP's `GET /api/v1/issues` endpoint, and deserialises the response body into `List<Map<String, Object>>` using a `ParameterizedTypeReference` — the correct Spring pattern that avoids raw type casts. A 10-second timeout is applied. If anything goes wrong, the

error is caught with `onErrorResume` and converted to a structured error map. The exception never propagates to the controller.

5. **Response.** The result flows back up through the handler (which adds the tool name and item count), through the dispatch service (which wraps it in `McpToolCallResponse.success`), through the controller (which serialises to JSON), and back to Claude. The entire pipeline is **non-blocking** — every method returns a `Mono<T>`. No thread is held waiting for a network response.

5. The Three Clients and What They Know

5.1 CibapClient — Five Services, One Client

`CibapClient` is the most complex client because it talks to five different services, not one. Each service has its own base URL stored in `McpProperties.cibapServiceUrls` — a `Map<String, String>` that maps service names like `"fraud-detection"` to URLs like `http://localhost:8082`. The client has a `urlFor(service)` helper that looks up the right URL for each call, then constructs a fresh `WebClient` instance for that base URL using an injected `WebClient.Builder`.

The endpoints it calls are mostly Spring Boot Actuator endpoints that all five services expose automatically:

Actuator Endpoint	Used For
`GET /actuator/health`	Service health status (UP/DOWN, sub-components)
`GET /actuator/prometheus`	Raw Prometheus metrics text — summarised by the handler
`GET /actuator/logfile`	Log file content — filtered and tailed by the handler
`GET /actuator/info`	Build and environment metadata

For Kafka status and recent deployment history, it calls custom REST endpoints on the orchestration service that have not been implemented yet. Rather than failing hard, the client catches those connection errors and returns a `Map` with `"note"` and `"hint"` keys explaining what endpoint needs to be added — so Claude can still convey useful information to the user.

> **Pod Status:** Since Kubernetes API access is not available in local development, `getPodStatus` immediately returns a static informational map without making any HTTP call at all. In a K8s environment this would query the Kubernetes API Server for live pod details.

5.2 CsipClient — Support Intelligence

`CsipClient` talks to one fixed base URL — port 8092 — and maps each of its 10 methods directly to CSIP's FastAPI routes:

MCP Tool	HTTP Call to CSIP
`csip_list_monitored_apps`	`GET /api/v1/apps`
`csip_register_app`	`POST /api/v1/apps`
`csip_get_issues`	`GET /api/v1/issues`
`csip_get_issue_detail`	`GET /api/v1/issues/{issueId}`
`csip_trigger_fix_retry`	`POST /api/v1/issues/{issueId}/escalate`
`csip_escalate_issue`	`POST /api/v1/issues/{issueId}/escalate`
`csip_get_audit_log`	`GET /api/v1/audit`
`csip_get_fix_pipeline_status`	`GET /api/v1/system/pipelines`
`csip_get_self_improvement_proposals`	Placeholder — not in CSIP backend yet
`csip_trigger_self_improvement`	Placeholder — not in CSIP backend yet

5.3 CtipClient — Test Intelligence

`CtipClient` talks to port 8091. Two design decisions stand out. First, `ctip_create_test` uses a **30-second timeout** instead of the standard 10 seconds, because generating a test case from a natural language scenario requires CTIP to make its own LLM call internally. Second, `ctip_run_test` (run a single test) has no dedicated endpoint in CTIP because CTIP's execution model is fundamentally suite-based — the client tunnels single-test execution through the suite runner by placing one test case ID in the `test_case_ids` array of the payload.

MCP Tool	HTTP Call to CTIP
`ctip_list_test_cases`	`GET /api/v1/test-cases`
`ctip_search_tests`	`GET /api/v1/search?q=...`
`ctip_create_test`	`POST /api/v1/test-cases` *(30s)*

MCP Tool	HTTP Call to CTIP
`ctip_run_test`	`POST /api/v1/runs/suite/default`
`ctip_run_suite`	`POST /api/v1/runs/suite/{service}`
`ctip_get_run_result`	`GET /api/v1/runs/suites` (filtered by ID)
`ctip_get_suite_result`	`GET /api/v1/runs/suites/{suiteRunId}`
`ctip_get_deployment_gate`	`GET /api/v1/gate/{service}/{commit}`
`ctip_get_coverage`	`GET /api/v1/coverage?service=...`
`ctip_get_flaky_tests`	`GET /api/v1/test-cases?status=flaky`
`ctip_get_scout_activity`	`GET /api/v1/scout/observations`
`ctip_get_self_improvement_proposals`	`GET /api/v1/self-improvement/proposals`

6. Configuration: The Single Source of Truth

`application.yaml` is the centralised configuration file. It sets the server port to 8095, declares all seven downstream service URLs under the `mcp.*` namespace, enables dev auth bypass, configures Resilience4j circuit breakers for each downstream system, exposes Spring Actuator endpoints, and sets logging levels.

```yaml
server:
  port: 8095

mcp:
  dev-bypass-auth: true
  csip-base-url: http://localhost:8092
  ctip-base-url: http://localhost:8091
  cibap-base-url: http://localhost:8081
  cibap-service-urls:
    customer-agent:  http://localhost:8081
    fraud-detection: http://localhost:8082
    loan-prescreen:  http://localhost:8083
    notification:    http://localhost:8084
    orchestration:   http://localhost:8085

resilience4j:
  circuitbreaker:
    instances:
      csip:  { slidingWindowSize: 10, failureRateThreshold: 50, waitDurationInOpenState: 30s
      ctip:  { slidingWindowSize: 10, failureRateThreshold: 50, waitDurationInOpenState: 30s
      cibap: { slidingWindowSize: 10, failureRateThreshold: 50, waitDurationInOpenState: 30s
```

`McpProperties` is the typed Java representation of those `mcp.*` YAML keys. Spring Boot's `@ConfigurationProperties(prefix = "mcp")` annotation automatically populates all fields from the YAML at startup. No client ever hardcodes a port number — they always ask `McpProperties` for the URL they need.

7. Security and Error Handling

`SecurityConfig` reads the `devBypassAuth` flag from `McpProperties`. In local development this flag is `true`, so the WebFlux security filter chain calls `permitAll()` on all exchanges and disables CSRF. No token is required. In production the flag would be `false`: the chain requires a valid Bearer JWT on all non-actuator paths, validated against a configured OAuth2 issuer URI.

`GlobalExceptionHandler` is annotated with `@RestControllerAdvice`, which registers it as a global exception interceptor across all controllers. It catches `IllegalArgumentException` — thrown by the dispatch service's default branch when an unrecognised tool name arrives — and converts it to a clean 400 JSON response with a structured `error` field, rather than letting Spring emit a generic error page.

8. The Tool Catalogue

`ToolCatalog` is a static registry — a `@Component` bean that constructs all 30 `McpToolDefinition` records at Spring startup and stores them in a list. Each definition carries four fields:

Field	Purpose
`name`	The exact string Claude sends in the tool call request — e.g. `csip_get_issues`
`description`	A paragraph written *for Claude to read*, explaining what the tool does, when to use it, and what the result looks like
`inputSchema`	A JSON Schema-style map specifying parameter names, types, defaults, enums, and whether each parameter is required
`readOnly`	`true` if the call has no side effects; `false` if it creates, modifies, or triggers something

Claude uses the `description` to decide *when* to call a tool and `inputSchema` to know *how* to populate the arguments. The `readOnly` flag lets Claude know whether a tool is safe to call in a read-only context versus requiring explicit user intent.

9. Full Tool Inventory (30 Tools)

CIBAP Tools — 8 tools

Tool Name	Description	Read Only
`cibap_health_check`	Health status of one or all five CIBAP services	Yes
`cibap_get_metrics`	Prometheus metrics summary for a service	Yes
`cibap_get_logs`	Filtered and tailed log lines for a service	Yes
`cibap_get_kafka_status`	Kafka topic and consumer group status	Yes
`cibap_get_recent_deployments`	Deployment history within N hours	Yes
`cibap_get_pod_status`	Kubernetes pod status for a service	Yes
`cibap_platform_summary`	Health of all services in one consolidated call	Yes
`cibap_get_service_info`	Metadata and static info about a service	Yes

CSIP Tools — 10 tools

Tool Name	Description	Read Only
`csip_list_monitored_apps`	List applications being monitored by CSIP	Yes
`csip_register_app`	Register a new application for autonomous monitoring	No
`csip_get_issues`	Query open, resolved, or all issues with filters	Yes
`csip_get_issue_detail`	Full detail for a specific issue including timeline	Yes

Tool Name	Description	Read Only
`csip_trigger_fix_retry`	Retry the automated fix pipeline for an issue	No
`csip_escalate_issue`	Escalate an issue for human engineering review	No
`csip_get_audit_log`	Full audit trail of all CSIP automated actions	Yes
`csip_get_fix_pipeline_status`	Status of active automated fix pipelines	Yes
`csip_get_self_improvement_proposals`	Pending self-improvement proposals from CSIP	Yes
`csip_trigger_self_improvement`	Apply a self-improvement proposal to CSIP	No

CTIP Tools — 12 tools

Tool Name	Description	Read Only
`ctip_list_test_cases`	List test cases with optional service and status filters	Yes
`ctip_search_tests`	Semantic search over test cases using natural language	Yes
`ctip_create_test`	Generate a test case from a natural language scenario (30s)	No
`ctip_run_test`	Execute a single test case via suite runner	No
`ctip_run_suite`	Run the full test suite for a given service	No
`ctip_get_run_result`	Result of a specific test run by ID	Yes
`ctip_get_suite_result`	Aggregated pass/fail result of a suite run	Yes

Tool Name	Description	Read Only
`ctip_get_deployment_gate`	Gate decision (pass/fail) for a CI/CD commit	Yes
`ctip_get_coverage`	Test coverage analysis for a service	Yes
`ctip_get_flaky_tests`	Tests with inconsistent pass/fail history	Yes
`ctip_get_scout_activity`	Web scout observations and newly discovered scenarios	Yes
`ctip_get_self_improvement_proposals`	CTIP self-improvement proposals by status	Yes